HeatSheet Language Specification

Yanni Kakouris Asa Shepard

May 14, 2025

Introduction

Managing a complete roster of track and field athletes is both time-consuming and difficult for coaches. Each athlete has many events they can run, personal records (PRs) in these events, and a few best events for scoring points at meets. For coaches, determining where athletes can perform best can be the difference between winning a meet or not. However, coaches tend to rely on intuition to determine the best event(s) for their athletes, which can be suboptimal in many cases.

Our language, HeatSheet, intends to streamline this process for coaches. By managing the roster under the hood, computing the expected points for an athlete in an event, and determining their comparative advantage, HeatSheet will complete all of the hard work for coaches. Given that many coaches aren't too technical or familiar with programming languages, our language will be designed to be as simple as possible. It won't require advanced syntax or programming knowledge to understand. Also, it will allow for coaches to overwrite our calculations if they strongly believe that an athlete should compete in a certain event (i.e. based on practice performance or gut feeling). The outputted PDF files will contain all the information a coach needs for a given athlete, roster, or competition, formatted in LaTeX.

Design Principles

HeatSheet will have a "coach-first" design. The language is built for track and field coaches, not programmers. Syntax and semantics prioritize readability, clarity, and domain familiarity. Whitespace will be entirely ignored in HeatSheet to allow coaches to format their programs as they see fit. The language will be declarative, so coaches will be able to describe what they want, not how to do it step by step. HeatSheet will compute mostly under-the-hood, making it easy for coaches to quickly get the information they need. Athletes, rosters, and meets are modeled as composable objects, so users can easily adapt the same logic across different scenarios or competitions. Ultimately, HeatSheet will be simple enough for dual meet planning, but powerful enough to handle championship meets or team selection for nationals if necessary.

Examples

Example 1:

```
let roster Williams;

let athlete: YanniKakouris,
  events: 100m, 200m, 4x100m,
  prs:
    100m: 11.01,
    200m: 22.42,
  maxEvents: 1;

add YanniKakouris to Williams;
output roster Williams;
```

Listing 1: HeatSheet Language Example 1

Example 1 defines a roster `Williams` and an athlete `YanniKakouris` with the specified events and personal records, as well as an optional tag "maxEvents". It then adds `YanniKakouris` to the `Wililams` roster. Finally, it ouputs the roster to a PDF generated using LaTeX.
Example 2:

```
let meet: StateChampionships,
  events: 100m, 200m, 400m, 4x100m, 4x400m,
  scoring: 1st: 10, 2nd: 8, 3rd: 6, 4th: 5,
           5th: 4, 6th: 3, 7th: 2, 8th: 1,
  maxEntries: 4;

optimize Williams for StateChampionships;
```

Listing 2: HeatSheet Language Example 2

Example 2 defines a meet titled `StateChampionships` with the specified events, scoring system, and max events per athlete. It then optimizes the entire `Williams` roster, placing each athlete into the best possible event to maximize score for `StateChampionships`. This optimize command will generate a LaTeX file and PDF showing where to place each athlete.

Example 3:

```
let athlete: AsaShepard,
  events: 400m, 800m, 4x400m,
  prs:
    400m: 48.68,
    800m: 1:55.04;
add AsaShepard to Williams;

let athlete: AmherstMammoth,
  events: 400m, 800m,
  prs:
    400m: 49.00,
    800m: 1:56.00;
add AmherstMammoth to Amherst;

add Amherst to StateChampionships;

optimize Williams for StateChampionships;

remove AsaShepard from Williams;
remove YanniKakouris from Williams;
```

Listing 3: HeatSheet Language Example 3

Example 3 defines another athlete, labeled `AsaShepard`. It then defines an athlete `AmherstMammoth` and places that athlete on the `Amherst` roster. It then includes them as a competitor at `StateChampionships`. Finally, it removes `AsaShepard` and `YanniKakouris` from the `Williams` roster.

Language Concepts

HeatSheet is built on a set of domain-specific objects, all composed from a small number of primitive types. These primitives include strings, integers and floats, booleans, and lists. These primitives form the foundation for the language's structured objects. The first is an athlete, a user-defined object that combines identifiers, eligible events, personal records, and other optional data. A meet is a composite structure that bundles events together with a scoring system and global constraints. Coaches must understand what makes up each of these objects, which they already know from their experience coaching.
Coaches can impose constraints to model the realities of competition, such as limiting an athlete to four events, even if it may not be theoretically optimal. These goals shape how the system evaluates possible lineups.

Formal Syntax

HeatSheet reads like a set of natural-language commands. A file is just a list of statements, one per line. It is enough for a coach to read, edit, and rerun without too much formal punctuation or brackets. Semicolons mark the end of a statement, allowing for all whitespace to be ignored.

```
<program>              ::= <statement> ;+

<statement>            ::= <roster-declaration>
                         | <athlete-declaration>
                         | <athlete-update>
                         | <set-pr>
                         | <meet-declaration>
                         | <meet-add>
                         | <meet-remove>
                         | <roster-add>
                         | <roster-removal>
                         | <output>
                         | <optimize>
                         | <duplicate>
                         | <set-optimize-type>
                         | <force-athlete>
                         | <free-athlete>

<roster-declaration>   ::= let roster <identifier>
                         | let roster <identifier>,
                           athletes: <athlete-list>

<athlete-declaration> ::= let athlete <identifier>,
                           events: <event-list>,
                           prs: <pr-list>
                         | let athlete <identifier>,
                           events: <event-list>,
                           prs: <pr-list>,
                           maxevents: <int>

<athlete-update>       ::= update athlete <identifier>,
                           events: <event-list>
                         | update athlete <identifier>,
                           events: <event-list>,
                           prs: <pr-list>
                         | update athlete <identifier>,
                           events: <event-list>,
                           prs: <pr-list>,
                           maxevents: <int>

<set-pr>               ::= set <athlete> to <time> in <event>

<meet-declaration>     ::= let meet <identifier>,
                           events: <event-list>,
                           scoring: <scoring-list>
                         | let meet <identifier>,
                           events: <event-list>,
                           scoring: <scoring-list>,
                           teams: <roster-list>
                         | let meet <identifier>,
                           events: <event-list>,
                           scoring: <scoring-list>,
                           teams: <roster-list>,
```

```
                            maxEntries: <int>

<meet-add>              ::= include <roster> in <meet>

<meet-remove>           ::= exclude <roster> from <meet>

<roster-add>            ::= add <athlete> to <roster>

<roster-removal>        ::= remove <athlete> from <roster>

<output>                ::= output <roster>
                          | output <meet>
                          | output <roster> to <path>
                          | output <meet> to <path>

<optimize>              ::= optimize <roster> for <meet>

<duplicate>             ::= duplicate <athlete> to <identifier>
                          | duplicate <roster> to <identifier>
                          | duplicate <meet> to <identifier>

<set-optimize-type>     ::= set optimization type as <optimization-type>

<force-athlete>         ::= force <athlete> to <event>

<free-athlete>          ::= free <athlete> from <event>

<athlete-list>          ::= <athlete> , <athlete>*
<roster-list>           ::= <roster> , <roster>*
<event-list>            ::= <event> , <event>*
<pr-list>               ::= <pr> , <pr>*
<scoring-list>          ::= <score-entry> , <score-entry>*

<pr>                    ::= <event> : <time>
<score-entry>           ::= <place> : <int>
<place>                 ::= <int> <suffix>
<time>                  ::= <float>
                          | <int> : <float>
                          | <int> : <int> : <float>

<path>                  ::= (<letter> | <digit> | "." | "/" | "_" | "-")+

<event>                 ::= <identifier>
<athlete>               ::= <identifier>
<roster>                ::= <identifier>
<meet>                  ::= <identifier>

<optimization-type>     ::= basic | simulation

<identifier>            ::= (<letter> | <digit>)*
<float>                 ::= <int> . <int>
<int>                   ::= <digit>+
<digit>                 ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<letter>                ::= a-z | A-Z
<suffix>                ::= st | nd | rd | th
```

Semantics

Primitive values include strings for names, integers, floats, time literals (stored internally as seconds with millisecond precision), booleans, and ordered lists. These values combine to form the four key objects: an *Athlete* (a name, eligible events, and a map from event to personal record), a *Roster* (a named collection of athletes), a *Meet* (its events, scoring table, and global constraints such as the maximum events per athlete), and an *Optimization*. All objects are immutable once declared. Only the references inside rosters and meets change when coaches add or remove athletes.

Evaluation proceeds top-to-bottom. The interpreter builds an in-memory catalogue as it reads. Each `add`, `remove`, or `force` mutates that catalogue in place. When it encounters `optimize Roster for Meet`, HeatSheet computes expected points from personal records, searches the legal assignment space while respecting every explicit `force`, and selects the best lineup. Results are written to a PDF using LaTeX, with the path echoed to the console; there are no other side-effects. An unknown identifier aborts execution with a clear error message, whereas non-fatal issues only generate warnings.

Semantics Table

Syntax	Abstract Syntax	Meaning
`let athlete: i, events: e1, e2, prs: p1, p2`	`Athlete of AthleteDeclaration`	Declares a new athlete *i* with a list of eligible events and personal records. Optionally includes a max events cap.
`update athlete: i, events: e1, prs: p1`	`AthleteUpdate of AthleteUpdate`	Updates athlete *i*'s events, PRs, and optionally max events. Errors if *i* is undeclared.
`set i to t in e`	`PRChange of SetPR`	Sets or updates the personal record for athlete *i* in event *e* to time *t*.
`let roster r`	`Roster of RosterDeclaration`	Declares a new team roster named *r*. It starts empty unless athletes are listed.
`add a to r`	`RosterAdd of RosterAdd`	Adds athlete *a* to roster *r*. Both must be previously declared.
`remove a from r`	`RosterRemoval of RosterRemoval`	Removes athlete *a* from roster *r*. Safe to call even if not present.
`let meet: m, events: e1, scoring: s`	`Meet of MeetDeclaration`	Declares meet *m* with events and scoring rules. May also include teams and max entries.
`optimize r for m`	`Optimize of Optimize`	Optimizes athlete assignments from roster *r* to maximize score in meet *m*.
`force a to e`	`ForceAthleteToEvent of ForceAthlete`	Forces athlete *a* to compete in event *e*, overriding optimization.
`free a from e`	`FreeAthleteFromEvent of FreeAthlete`	Unforces athlete *a* from event *e*, restoring default assignment logic.
`output r`	`RosterShow of RosterToShow`	Generates LaTeX and PDF output for roster *r*. Optionally accepts a file path.
`output m`	`MeetShow of MeetToShow`	Generates LaTeX and PDF output for meet *m*, showing all teams and entries.
`<i>`	`Identifier`	A name used for athletes, rosters, events, or meets. Internally represented as a string.
`<t>`	`Time`	A race time. Can be a float (e.g., `11.01`), a minute:second pair (e.g., `1:55.04`), or hour:minute:second. Normalized to seconds.
`<n>`	`int`	A non-negative integer used for max events, entry caps, placements, etc.
`<b>`	`bool`	A boolean flag (e.g., internal or future constraint handling).
`<list>`	`List of primitives`	A comma-separated list of identifiers, events, scores, or times.

Remaining Work

To make it easier for coaches, our language would benefit from a GUI, similar to R-Studio or STATA, that allows coaches to see the current environment they created. It would also be helpful to save athletes and rosters in a custom file so that coaches can save/load certain rosters. Implementing these two features was not feasible given the time constraint for this project, as well as being limited to F#.

Another improvement would be to implement a connection to a track database, like TFRRS or Athletic.net, that would allow coaches to quickly gather all information on their athletes without having to input their informatin manually. Since there is no complete database with a public API, this task is difficult and would require scraping their websites. Lastly, it is likely that some coaches would like more control over the optimization. It is difficult to know exactly what these features may be, but feedback from real coaches would help us determine helpful features to add.